

CineMatcha

AI/ML-Based Movie Recommendation System

PROJECT REPORT

A Python-based content recommendation system using
TF-IDF Vectorization and Cosine Similarity

Project Name	CineMatcha
Domain	Artificial Intelligence & Machine Learning
Programming Language	Python
Platform	Google Colab
Recommendation Type	Content-Based Filtering

Submitted as an AI/ML Project

Academic Year: 2026–27

2. Introduction

Movie recommendation systems help users discover films that match their interests. CineMatcha is a small AI/ML project that recommends movies based on the characteristics of a movie selected by the user. The project uses movie genres and descriptions as input features and applies text processing and similarity analysis to identify related movies.

The system follows a content-based filtering approach. It does not require a user's complete viewing history; instead, it compares the selected movie with the available movies in the dataset. This makes the project simple, transparent, and suitable for demonstrating the basic concepts of machine learning and natural language processing.

3. Problem Statement

With a large number of movies available, users may spend considerable time trying to decide what to watch. A simple recommendation system can reduce this effort by identifying movies that share similar content with a movie the user already likes.

Problem: How can movie information such as genre and description be used to automatically find and recommend similar movies?

4. Functional Requirements

- Display the list of movies available in the system.
- Accept a movie name from the user.
- Search for the selected movie in the dataset.
- Process movie genre and description.
- Generate TF-IDF feature vectors.
- Calculate similarity between movies.
- Sort movies according to their similarity score.
- Display the top five recommendations.
- Handle an invalid movie name with a suitable message.

5. Non-functional Requirements

- **Usability:** The program should be easy for a student/user to operate.
- **Performance:** Recommendations should be generated quickly for the available dataset.
- **Reliability:** Invalid movie names should not crash the program.
- **Maintainability:** Movie data and recommendation logic should be easy to update.
- **Portability:** The project should run in a standard Python environment or Google Colab.

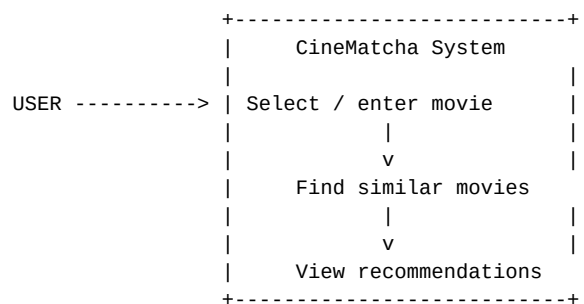
6. System Architecture

Layer	Component	Purpose
Input	User	Provides the movie name
Data	Movie Dataset	Stores titles, genres and descriptions
Processing	TF-IDF Vectorizer	Converts text into numerical features
ML	Cosine Similarity	Compares movie feature vectors
Recommendation	Ranking Logic	Selects the most similar movies
Output	Recommended Movies	Displays top 5 results

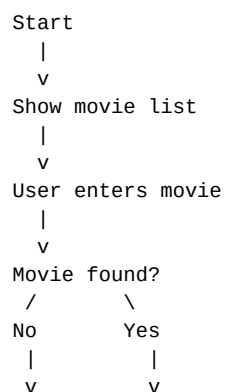
```
USER
|
v
MOVIE DATA
|
v
GENRE + DESCRIPTION
|
v
TF-IDF VECTORIZATION
|
v
COSINE SIMILARITY
|
v
RANK SIMILAR MOVIES
|
v
TOP 5 RECOMMENDATIONS
```

7. Design Diagrams

7.1 Use Case Diagram



7.2 Workflow Diagram



```

Error    Get movie features
message  |
          v
        TF-IDF conversion
          |
          v
    Calculate similarity
          |
          v
        Sort similarity
          |
          v
        Show top 5 movies
          |
          v
          End

```

7.3 Sequence Diagram

```

sequenceDiagram
    participant User
    participant System
    participant TF-IDF
    participant Similarity

    User->>System: --movie--
    System->>TF-IDF: --features--
    TF-IDF-->>System: <--vectors--
    System->>Similarity: <-----scores-----
    Similarity-->>System: --rank results
    System-->>User: <--results--

```

7.4 Class/Component Diagram

```

classDiagram
    class MovieData {
        +Movie Data
    }
    class RecommendationFunction {
        +Recommendation Function
    }
    class TFIDFVectorizer {
        +TF-IDF Vectorizer
    }
    class CosineSimilarity {
        +Cosine Similarity
    }
    class RecommendationOutput {
        +Recommendation Output
    }

    MovieData --> RecommendationFunction
    RecommendationFunction --> TFIDFVectorizer
    RecommendationFunction --> CosineSimilarity
    RecommendationFunction --> RecommendationOutput

```

7.5 ER Diagram (if storage used)

The current prototype keeps movie data in a Pandas DataFrame and does not use a persistent database. If database storage is added later, a Movie table can contain Movie_ID, Title, Genre and Description fields, while a User_Rating table can contain User_ID, Movie_ID and Rating.

8. Design Decisions & Rationale

Decision	Reason
Content-based filtering	Simple to implement and works without user history.
TF-IDF	Suitable for converting movie text into useful numerical features.
Cosine Similarity	Simple and effective for comparing text-based vectors.
Pandas DataFrame	Easy to store and process the project dataset.
Python + Scikit-learn	Provides readable code and standard ML tools.
Google Colab	Easy environment for running and demonstrating the project.

9. Implementation Details

The implementation is written in Python. A Pandas DataFrame stores the movie information. Genre and description are combined into a single feature string. Scikit-learn's TfidfVectorizer transforms these strings into numerical vectors. Cosine Similarity then creates a similarity matrix for the movies.

Core implementation flow:

```
movie features
  |
  v
TfidfVectorizer()
  |
  v
TF-IDF matrix
  |
  v
cosine_similarity()
  |
  v
similarity matrix
  |
  v
sort scores
  |
  v
top 5 recommendations
```

The recommendation function first checks whether the entered movie exists. If it is found, its similarity scores are retrieved and sorted from highest to lowest. The selected movie itself is skipped, and the next five movies are shown to the user.

10. Screenshots / Results

The following represents the expected console result after entering a movie. The exact similarity percentages depend on the dataset and feature text.

```
■ MOVIE RECOMMENDATION SYSTEM
=====

Available Movies:
• Inception
• Interstellar
• The Dark Knight
• The Matrix
• Iron Man
• Avengers Endgame
• Avatar
• Jurassic Park
• Titanic
```

- The Shawshank Redemption

Enter a movie you like: Inception

■ Recommended Movies

=====

1. The Matrix (similarity score)
2. Interstellar (similarity score)
3. Avatar (similarity score)
4. Iron Man (similarity score)
5. Jurassic Park (similarity score)

Result Interpretation

For a movie such as Inception, movies containing related action, science fiction, thriller, adventure, or related descriptive terms can receive higher similarity values. The model ranks these values and presents the highest matches first.

11. Testing Approach

Test Case	Input	Expected Result
Valid movie	Inception	Top 5 similar movies displayed
Valid movie	The Matrix	Top 5 similar movies displayed
Different genre	Titanic	Movies ranked using content similarity
Invalid movie	Batman 2099	Movie not found message
Extra spaces	Inception	Input is cleaned and processed

Testing focuses on normal movie inputs, invalid inputs, and minor input formatting differences. The objective is to ensure that the recommendation function produces results without terminating unexpectedly.

12. Challenges Faced

- Choosing useful movie features for comparison.
- Understanding how text needs to be converted into numerical data before similarity can be calculated.
- Handling movie names that are not present in the dataset.
- Keeping the recommendation logic simple enough to explain during a project presentation.
- Ensuring that the model ranks the similarity scores correctly.

13. Learnings & Key Takeaways

- Learned the basic working of a content-based recommendation system.
- Understood how TF-IDF can represent text numerically.
- Learned how Cosine Similarity is used to compare feature vectors.
- Practiced working with Pandas and Scikit-learn.
- Understood how an ML idea can be converted into a working Python program.
- Learned the importance of testing invalid and normal user inputs.

14. Future Enhancements

- Use a larger real-world movie dataset.
- Add user accounts, ratings, and watch history.
- Implement collaborative filtering.
- Combine content-based and collaborative filtering into a hybrid system.
- Add movie posters, cast, director, ratings, and trailers.
- Create an interactive interface using Streamlit or Flask.
- Add genre filters and improved movie search.
- Improve personalization using user preferences.

15. References

- Python documentation — Python programming concepts and standard libraries.
- Pandas documentation — DataFrame and data-processing concepts.

- Scikit-learn documentation — TfidfVectorizer and cosine similarity.
- General concepts of content-based recommendation systems.
- General Natural Language Processing (NLP) and TF-IDF concepts.

Project: CineMatcha | **Domain:** AI/ML | **Platform:** Google Colab